

Testing, Debugging, and Deployment

Given a scenario, Apex code, Apex trigger, or Apex test that is not executing as expected, apply techniques and tools to isolate and identify the root cause.

Table of Contents

-  [Performance Profiling](#)
-  [Client-Side Tools](#)
-  [Server-Side Tools](#)
-  [Profiling Performance Considerations](#)
-  [Developer Console](#)
-  [Examples](#)
-  [Scenarios and Solutions](#)
-  [Apex Tests](#)



Table of Contents

-  [Apex Test Scenario / Solution I](#)
-  [Apex Test Scenario / Solution II](#)
-  [Apex Test Scenario / Solution III](#)
-  [Apex Test Scenario / Solution IV](#)
-  [Apex Test Scenario / Solution V](#)
-  [Apex Test Scenario / Solution VI](#)



After studying this topic, you should be able to:

-  Describe techniques and tools that can be used to isolate and resolve issues.
-  Describe processes to identify and manage issues related to org performance.
-  Identify the techniques and tools that can be used for troubleshooting Apex code and tests.
-  Identify how to write proper tests to build robust applications using Apex.



Introduction

When **Apex classes**, **triggers**, **Visualforce controllers**, **controller extensions**, or **Apex tests** are not working as expected, or performance issues are encountered, **tools and techniques** are available to identify the **root causes**.

This topic will cover concepts and scenarios to determine the appropriate **methods**, **procedures**, and **considerations** for resolving issues generally caused by Apex.



Performance Profiling & Troubleshooting

Client-Side Tools

Client-side and network performance tools, such as Fiddler, can be utilized.

Server-Side Tools

Server-side tools, such as the Developer Console, can be used to analyze server-side application performance.

Profiling Considerations

Performance Profiling has considerations related to sharing, governor limits, load testing, etc.

Lightning Framework

When debugging Lightning component JavaScript code, debug mode can be enabled for specific users.

Troubleshooting

Debug logs can be generated using debug traces to identify and fix performance issues.

Performance Analysis

An application's performance can be analyzed using client-side and server-side tools.

Performance Profiling

Performance Profiling is a process of identifying and fixing application bottlenecks for optimal performance.



Performance Profiling

Table of Contents



Performance Profiling

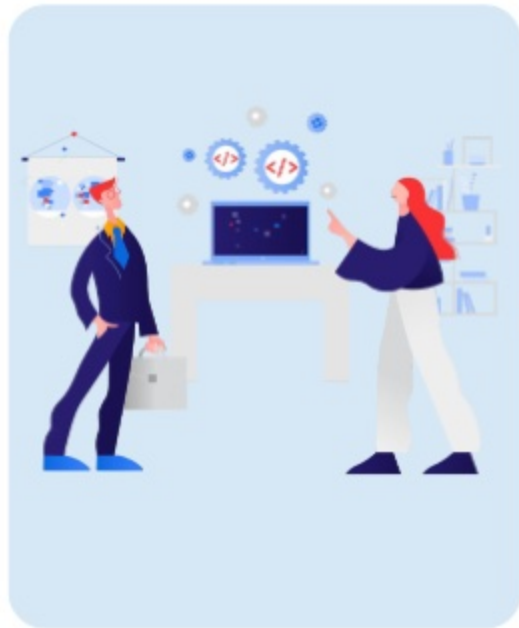
Salesforce provides **tools** that enable developers to **implement** application performance profiling.

◆ PROCESS

Performance profiling refers to the **process of identifying and fixing performance bottlenecks**, ensuring that applications are fast and scalable.

◆ TECHNIQUE

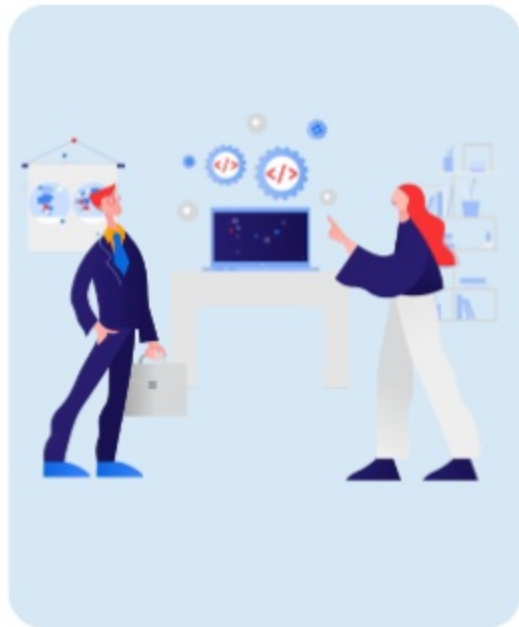
Performance profiling techniques can help **identify potential performance issues** and **apply best practices** to improve the performance of the application.



Performance Profiling

◆ TESTING

It can be **used with functional unit testing** to ensure that an application is coded to properly handle the large data volumes and/or complex sharing configurations.



Performance Profiling Methodology

A performance profiling **methodology** consists of the following **high-level steps**:



❖ STEP 1: SCOPE DEFINITION

Defining the **scope of profiling** by splitting scenarios into different functionalities



❖ STEP 2: ANALYSIS

Analyzing and measuring the **performance** of the application for each functionality



❖ STEP 3: IDENTIFICATION

Identifying **potential bottlenecks** and their root causes



❖ STEP 4: REMEDY

Fixing the **root causes** of the bottlenecks

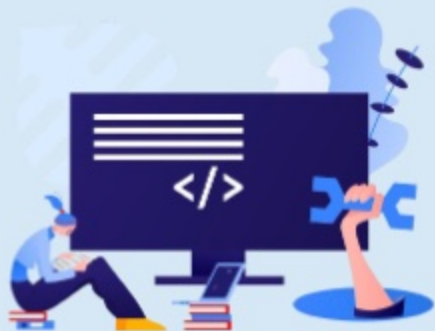


❖ STEP 5: RERUN

Comparing **results** and **repeating** the previous steps until the bottlenecks are removed

Analysis & Troubleshooting Process

The following **process** can be used for performance analysis and troubleshooting:



❖ WEB ANALYSIS TOOLS

Web analysis tools, such as Chrome Developer Tools, Firefox Developer Tools, or Fiddler, can be used to identify network issues, client-side problems, and resource optimization opportunities.

❖ DEBUG TRACE FLAGS

Debug traces can be set up for users and Apex classes; a debug level of **INFO** can be initially set and adjusted upward as necessary until the desired level of detail is achieved.

Analysis & Troubleshooting Process



◆ DEBUG LOGS

When Apex code is **run**, a debug log is **generated**.

◆ TIMELINE TAB

The Timeline tab of the **Execution Overview** panel in the Developer Console can be analyzed for either:

- **Duration**: To **identify processes** that take significantly **more time** than others.
- **Efficiency**: To **detect a sequence** of many short, rapidly repeated processes, which may indicate an **inefficient looping statement**.

Analysis & Troubleshooting Process



◆ TIMELINE TOOL

The Apex Timeline tool can be used for evaluating **larger log files**.

◆ QUERY PLAN

In **Developer Console**, the Query Plan tab can be used to analyze how SOQL queries are executed in the org and identify inefficient queries.

Client-Side Tools

Table of Contents



Lightning Debug Mode

The **Lightning Component Framework** executes in one of two modes: **production** or **debug**.

PRODUCTION MODE

The **default** execution code is **production**, which is optimized for **performance**. In this mode, the framework code is **optimized**, **minified**, and **obfuscated**, which reduces the size of the JavaScript code. Custom component code is not minified or obfuscated.



DEBUG MODE

When the debug mode is **enabled**, the framework **JavaScript code** is **not minified** and is easier to read and debug. However, Salesforce is **slower** for any user who has debug mode enabled.



Lightning Debug Mode

The Lightning Component framework provides a **mode** specialized for application code **debugging** and **troubleshooting**.

SELECT DEBUG USERS



It is possible to **enable** debug mode **only** for the users who are involved in debugging activity to avoid affecting the performance of Salesforce for other users in the org.

ENABLE DEBUG MODE



Debug mode can be enabled for **specific users** by navigating to '**Debug Mode**' under the '**Lightning Components**' section in Setup.

BANNER NOTIFICATION



Users who have debug mode **enabled** see a banner notification **once a week** while it is enabled.

Debug Mode

The debug mode can be **enabled** as well as **disabled** for **multiple** users.

Search: Debug

- Custom Code
 - Lightning Components
 - Debug Mode**
 - Environments
 - Logs
 - Debug Logs

Didn't find what you're looking for? Try using Global Search.

Debug Mode

Debug Mode Users

Enable debug mode to make it easier to debug JavaScript. Salesforce is slower for users who have debug mode enabled.

View: Custom View | Edit | Create New View

Enable **Disable**

☐	Full Name ↑	Debug Mode	Alias	Active	Role	Profile	Locale	Language
☐	Barao, Filia	☐	fbarao	☐	European Sales	Custom: Sales Profile	English (United States)	English
☐	Campbell, Mary	☐	mcampbel	☐	European Sales Manager	Custom: Sales Profile	English (United States)	English
☐	Chatter Expert	☐	Chatter	✓		Chatter Free User	English (United States)	English
☐	Gessner, Martin	☐	mgessner	✓	Manager	System Administrator	English (United States)	English
☐	Jones, Maria	✓	jsmith	✓	European Sales Manager	Extended Standard User	English (United States)	English
☐	Kowalski, Ligita	☐	lkowa	☐	European Sales	Custom: Sales Profile	English (United States)	English
☐	Sweeney, Beth	☐	bsweeney	☐	European Sales	Custom: Sales Profile	English (United States)	English

Debug mode can be enabled for specific users of the org who are involved in debugging framework JavaScript code.

Client-Side and Network Performance Tools

The following **tools** can be used for **analyzing**, **debugging**, and **troubleshooting** client-side and network performance of Salesforce applications.



CHROME DEVELOPER TOOLS

Chrome Developer Tools (a Chrome built-in tool) provide graphical representations of how the **time is spent** from the moment the **page request** is sent until the **page load** is complete.



FIREFOX DEVELOPER TOOLS

Firefox Developer Tools can be used to examine, edit, and debug **HTML**, **CSS**, and **JavaScript**.

Client-Side and Network Performance Tools



FIDDLER

Fiddler enables inspection and analysis of **web requests** and **responses** to identify **bottlenecks**.



CHARLES

Charles, like Fiddler, enables inspection and analysis of **web requests** and **responses** to identify **bottlenecks**.



WEBPAGETEST

WebPageTest performs **speed tests** based on various combinations of test **location**, **browser**, and **connection** speed.

Server-Side Tools

Table of Contents

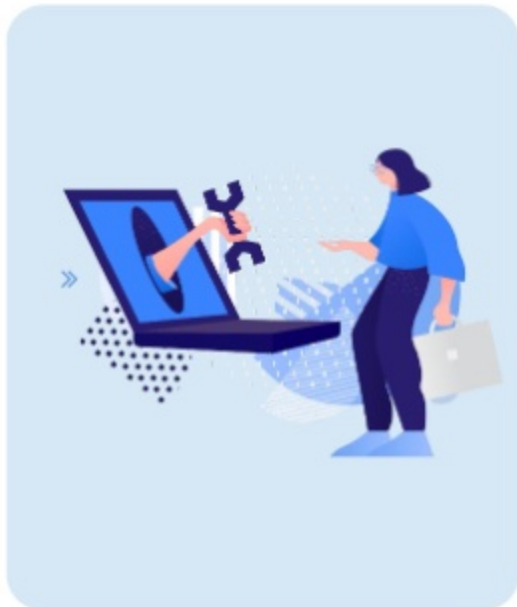


Developer Console Tools

The following **components** of the Developer Console can be used for analyzing **server-side** performance:

◆ EXECUTION OVERVIEW PANEL

The **Timeline tab** of the execution overview panel provides a **visual representation** of **how long** each type of process took.



Developer Console Tools

STACK TREE PANEL

The Stack Tree panel **contains** the following **tabs**:

❖ PERFORMANCE TREE

Provides a **top-down view** that shows the **amount of time** a specific section of code took to execute, groups **repeated statements** into a single total, and shows **number** of **iterations**.

❖ EXECUTION TREE

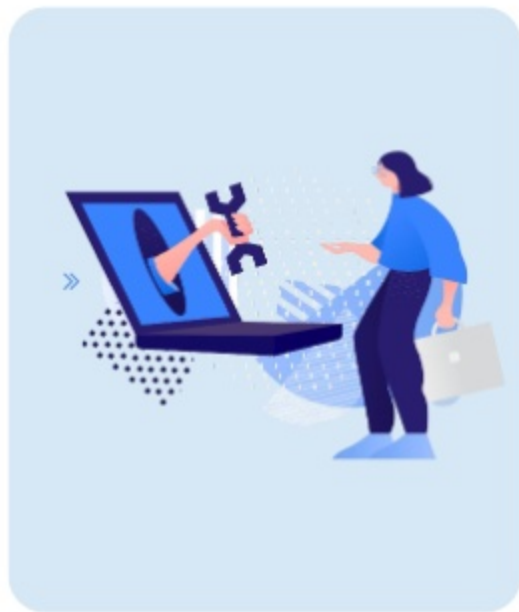
Provides a **top-down view** that shows the **amount of time** a specific section of code took to execute; it shows each **iteration** of a repeated statement in a **separate line**.



Developer Console Tools

◆ EXECUTION STACK

This tool provides a **bottom-up view** that shows the **amount of time** a **specific section** of code took to execute; it shows each **iteration** of a repeated statement in a **separate line**.



Other Server-Side Performance Tools

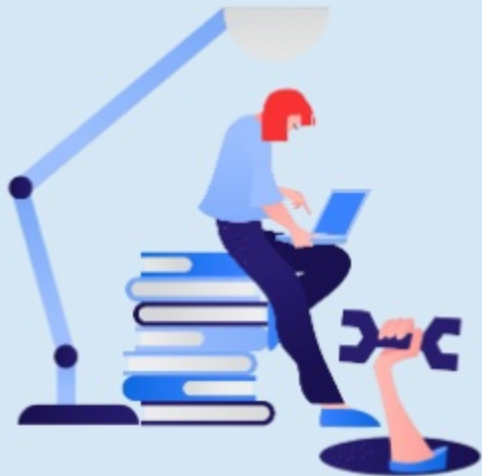
The following lists more **tools** that can be used for analyzing server-side **application performance**:

❖ TOOLING API

The Tooling API retrieves **log files** via **API calls** for further analysis with tools or code.

❖ APEX TIMELINE TOOL

- Uses the Tooling API to retrieve **debug logs** and show a **graphical representation** of events
- Extracts **SOQL queries**, **DML operations** and **Apex trigger** information
- Provides a representation of key data in **JSON format** that can be **parsed** with another tool for further analysis



Profiling Performance Considerations

Table of Contents



Considerations for Profiling Performance

The following are **considerations** related to profiling performance:



ROLE USERS

Users with different roles may be used when profiling performance to identify the impact of data volume and sharing calculation overheads.



GOVERNOR LIMIT

Governor limit **usage** can be **identified** using '**LIMIT_USAGE**' in the Execution Log panel.



BUSINESS PROCESS

Business processes **should not be** designed with the assumption that **only optimal test results** will be observed.



QUERY PERFORMANCE

Query performance should be **evaluated with a custom field without an index** to establish the query's **baseline performance**.



CUSTOMER SUPPORT

Salesforce Customer Support should be consulted before running any load testing, including stress, bandwidth, or concurrent user testing.

Performance Profiling Example

The following **Apex controller** and **Visualforce pages** have been created and are being reviewed for ways to **improve their performance**.

```
public class OppController {
    public list<AggregateResult> lstAgg = new list<AggregateResult>();
    public OppController() {
        lstAgg = [
            SELECT StageName, SUM(Amount) Amount, COUNT(id) Total
            FROM Opportunity WHERE isClosed = false AND StageName != 'Canceled' GROUP BY StageName
        ];
    }
    public list<OppAgg> getResults() {
        list<OppAgg> lstResult = new list<OppAgg>();
        for (AggregateResult ar: lstAgg) {
            OppAgg objOppAgg = new OppAgg(ar);
            lstResult.add(objOppAgg);
        }
        return lstResult;
    }
    class OppAgg {
        public Integer Total {get;set;}
    }
}
```

Continued...

Performance Profiling Example

```
public String StageName {get;set;}
public Decimal Amount {get;set;}
public OppAgg(AggregateResult ar) {
    Total = (Integer)ar.get('Total');
    StageName = (String)ar.get('StageName');
    Amount = (Decimal)ar.get('Amount');
}
}
}

<apex:page sidebar="false" controller="OppController">
    <apex:pageBlock title="Sales Aggregate">
        <apex:pageBlockTable value="{!Results}" var="ar">
            <apex:column headerValue="Stage Name" value="{!ar.StageName}"/>
            <apex:column headerValue="Total Amount" value="{!ar.Amount}"/>
            <apex:column headerValue="Number of Orders" value="{!ar.Total}"/>
        </apex:pageBlockTable>
    </apex:pageBlock>
</apex:page>
```

End of OppController

Visualforce page

Developer Console

The Developer Console is used below to **determine** and **analyze** the **events** that transpired in an Apex transaction.

The screenshot displays the Salesforce Developer Console interface with three main panels: Performance Tree, Execution Stack, and Execution Log. Annotations highlight specific data points and features.

Performance Tree: Shows the duration of various units. A callout points to the SOQL query unit, indicating it took 12.85ms to execute.

Unit	Duration	Heap	Iterations
Visualforce Page OppController	65.50	2357	1
Apex Class OppController	2.08	305	6
SELECT StageName, SUM(Amount) Amount, COUN...	12.85	0	0
OppController	0.02	0	0

Execution Stack: Provides a bottom-up view of execution time. A callout points to the Visualforce page unit, indicating it took 65.5ms to load.

Unit	Duration	Heap
OppController	0.02	0
Visualforce Page OppController	65.50	2357
/apex/OppController	65.57	2357

Execution Log: Lists events with timestamps, event names, and details. A callout notes that the filter can be used to identify events associated with the DB category.

Timestamp	Event	Details
16:32:56.009	SOQL_EXECUTE...	[6][Aggregations:0]SELECT StageName, SUM(Amount) A
16:32:56.022	SOQL_EXECUTE...	[6][Rows:6

The filter field is set to `{ "category": "DB" }`.

Continued...

Developer Console

The *Timeline* tab of the *Execution Overview* panel shows that the database query (DB) took longer to complete compared to the other processes.

These panels can be used to analyze code performance. This window can be opened by navigating to 'View Log Panels' in the Debug menu.

Execution Stack

Unit	Time (ms)	Count	Count per Unit
Visualforce Page OppController	65.50	2357	1
Apex Class OppController	2.08	305	6
SELECT StageName, SUM(Amount) Amount, COUN...	12.85	0	1
OppController	0.02	0	1

Execution Overview

Save Order Limits **Timeline** Executed Units

Category	Millis	%	16:32:56.000	16:32:56.010	16:32:56.020	16:32:56.030	16:32:56.040
APEX_CODE	2.08	3.27%					
WORKFLOW	0.00	0.00%					
DB	12.85	20.21%					
VALIDATION	0.00	0.00%					

Scale: ☒ 0.01 Seconds ☐ 0.1 Seconds ☐ 1 Seconds ☐ 10 Seconds ☐ Minutes

Select Panels

- ☒ Stack Tree
- ☒ Execution Stack
- ☒ Execution Log
- ☐ Source
- ☐ Source List
- ☐ Variables
- ☒ Execution Overview

Logs, Tests, and Problems

Developer Console

Table of Contents



Obtaining Summary Information

The Developer Console tool provides a **summary of events** that took place in a transaction.

EXECUTION OVERVIEW



The **Execution Overview** panel at the bottom of the Log Inspector in the Developer Console can be used to **obtain summary information about processes**.

EXECUTION LOG



Although the **Execution Log** provides similar information, it is quicker to access the Execution Overview panel to **view the necessary information without applying filters**.

Obtaining Summary Information

The **panel** contains four tabs:



Save Order



Limits



Timeline



Executed Units

Save Order

The Save Order tab displays a **color-coded** timeline of **DML actions** and **save order elements**, such as before and after triggers, validation rules, record-triggered flows, etc.



COLOR-CODED

Different colors are used to differentiate between **save order elements**.



ACTION DETAILS

Details about the element can be accessed by **clicking the boxcar**, which is a batch of actions that represents it.

Save Order Tab

The Save Order tab can be **accessed** through the **Execution Overview** panel.

The screenshot displays the Salesforce Execution Overview panel. The 'Execution Overview' tab is selected, and the 'Save Order' sub-tab is active. A yellow callout box highlights the 'Save Order' tab with the text: 'The Save Order tab in the Execution Overview panel quickly provides information about the save order of execution associated with the transaction.'

Stack Tree

Execution Tree | Performance Tree

Unit	Duration	Heap
/services/data/v49.0/to		

Execution Stack

Unit	Duration
Flow:Account	0.02
Payment Status...	0.05
Transaction Stat...	6.45
Workflow	6.59

Enter Apex Code

```
1 List<Opportunity> opps = [SELECT Id, Name, StageName FROM Opportunity];
2 List<Opportunity> oppsToUpdate = new List<Opportunity>();
3 Account a = new Account(Name='Cosmic Enterprises');
4 update oppsToUpdate;
5 insert a;
```

Execution Log

Timestamp	Event	Details
17:39:31:001	USER_INFO	[EXTERNAL] 0052v00000YeuUv ayandyan+force@gmail.
17:39:31:001	EXECUTION_ST	

Execution Overview

Save Order | Limits | Timeline

Subject			
Account Insert	Apex Trigger InefficientQueryLookupAccountTrigger	Validation	Workflow

Limits

The Limits tab shows the **overall system limits** by name and amount used.



LIMIT DETAILS

It displays the **name** of the limit, amount used at the **point of execution**, amount used at **completion**, and the **total amount** available.



DEBUG LEVEL

The profiling debug level must be set to '**Finest**' to obtain this information.

Limits Tab

The Limits tab can be accessed through **Execution Overview** panel.

The screenshot displays the Salesforce Apex Limits tab. At the top, there are two main panels: 'Stack Tree' and 'Execution Log'. The 'Stack Tree' panel includes an 'Execution Tree' sub-panel showing a tree view of the execution stack, and an 'Execution Stack' sub-panel showing a table of execution units. The 'Execution Log' panel shows a table of execution events. A modal window titled 'Enter Apex Code' is open, displaying a list of Apex code lines. The 'Execution Overview' panel is at the bottom, featuring a 'Limits' tab and a table of system limits. A yellow callout box points to the 'Limits' tab with the text: 'The Limits tab shows the name of the limit and the information related to the limit.'

Stack Tree

Execution Tree Performance Tree

Unit Duration Heap

/services/data/v49.0/to

Execution Stack

Unit Duration

Flow:Account 0.02

Payment Status... 0.04

Transaction Stat... 5.71

Workflow 5.84

Execution Log

Timestamp Event Details

17:48:05:003 USER_INFO [EXTERNAL]0052v00000YeuUv|ayandyan+force@gmail.

17:48:05:003 EXECUTION ST

Enter Apex Code

```
1 List<Opportunity> opps = [SELECT Id, Name, StageName FROM Opportunity];
2 List<Opportunity> oppsToUpdate = new List<Opportunity>();
3 Account a = new Account(Name='Cosmic Enterprises');
4 update oppsToUpdate;
5 insert a;
```

Execution Overview

Save Order **Limits** Executed Units

Limit	Used so far	Request Total	Total Available
SOQL	0	1	100
AGGS	0	0	300
SOQL_ROWS	0	532	50000
DML	0	1	150
DML_ROWS	0	1	10000

Timeline

The Timeline tab provides a quick **visual representation** of the amount of **time** taken by **each process**.



EVENT DURATION

It contains information about the **type** of process, **time** in milliseconds, and the **percentage** of time taken for the entire request.

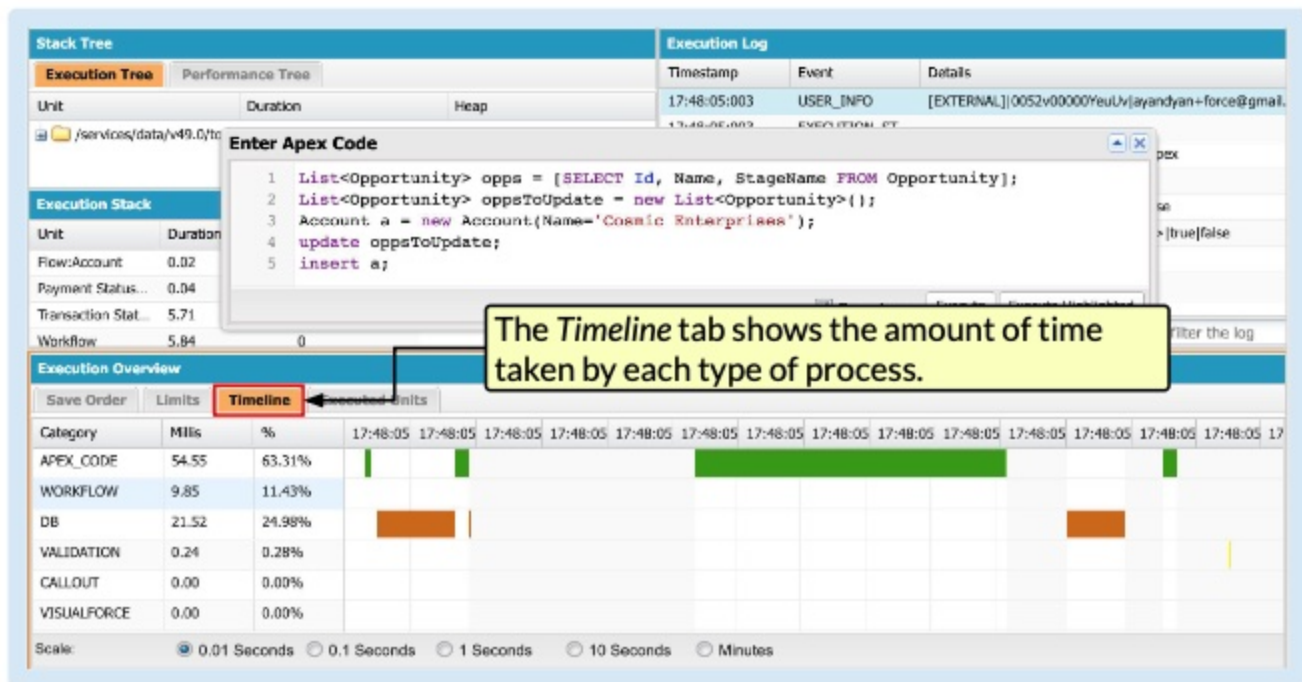


SCALE OPTION

The Scale option can be used to **increase** or **decrease** the **time interval** represented in the time duration columns.

Timeline Tab

The Timeline tab can be accessed through the **Execution Overview** panel.



Executed Units

The Executed Units tab displays the **system resources** used by **each item** in the process.



METRICS

It provides information about the **process name**, its **total**, **average**, **maximum**, and **minimum duration**, the number of **times** it was called, **heap size**, **query type**, and the **total**, **average**, **maximum**, and **minimum** number of **records changed**.



SORTED

Information can be **sorted by any specific column**.



FILTERED

Buttons at the bottom of the tab can be used to **filter information by item type**.

Executed Units Tab

The Executed Units tab can be accessed through the **Execution Overview** panel.

The screenshot displays the Salesforce Developer Console interface. The **Execution Overview** panel is active, with the **Executed Units** tab selected. A callout box points to this tab, stating: "The Executed Units tab provides information such as the number of times a method got called and the number of rows returned by a query."

Stack Tree

Unit	Duration	Heap
/services/data/v49.0/...		

Execution Stack

Unit	Duration
Flow:Account	0.02
Payment Status...	0.04
Transaction Stat...	5.71
Workflow	5.84

Enter Apex Code

```
1 List<Opportunity> opps = [SELECT Id, Name, StageName FROM Opportunity];
2 List<Opportunity> oppsToUpdate = new List<Opportunity>();
3 Account a = new Account(Name='Cosmic Enterprises');
4 update oppsToUpdate;
5 insert a;
```

Execution Overview

What	Name	Sum	Avg	Max	Min	Count	Heap	Query Type	Sum rows	Avg rows	Max rows	Min row
Trigger	Apex Trigg...	1.94	1.94	1.94	1.94	1	16	n/a	n/a	n/a	n/a	n/a
Workflow	Transactio...	5.71	5.71	5.71	5.71	1	0	n/a	n/a	n/a	n/a	n/a
Method	execute_a...	255.04	255.04	255.04	255.04	1	66567	n/a	n/a	n/a	n/a	n/a
Method	/services/d...	256.94	256.94	256.94	256.94	1	0	n/a	n/a	n/a	n/a	n/a
Method	Flow:Acco...	48.41	24.21	48.40	0.02	2	0	n/a	n/a	n/a	n/a	n/a
Validation	Validation:...	0.24	0.24	0.24	0.24	1	0	n/a	n/a	n/a	n/a	n/a
Insert	Account In...	208.99	208.99	208.99	208.99	1	8	n/a	1	1	1	1

Show: Methods | Queries | Workflow | Callouts | DML | Validations | Triggers | Pages

Examples

Table of Contents



Debug Logging Example

An **Apex trigger**, which executes **static methods** belonging to two different classes, is not producing the expected results.

```
trigger ContactTrigger on Contact (after update) {  
  
    System.debug('ContactTrigger executed!');  
    DemoClass.someMethod();  
    UtilityClass.someMethod();  
}
```

The developer needs to figure out where in the process the logic is failing and aims to trace the processes **executed** or **invoked** by the *DemoClass* class first.

Debug Logging Example

By using **trace flags**, a debug level can be specified for the *DemoClass* to **increase** the **log verbosity** for transaction details associated with the class.

[illegible]

Tracking DML Example

A developer is building a custom functionality in the org that involves an **Apex class** and **triggers** that perform **DML statements** at various points in the execution process.

```
trigger ContactTrigger on Contact (after insert) {
    List<Coupon__c> coupons = new List<Coupon__c>();
    for (Contact c : Trigger.New) {
        if (c.Has_Coupon__c == TRUE) {
            Coupon__c coupon = new Coupon__c();
            coupon.Contact__c = c.Id;
            coupons.add(coupon);

            // other logic below...
        }
    }
    insert coupons;
}

trigger CouponTrigger on Coupon__c (after insert) {
    for (Coupon__c c : Trigger.New) {
        UtilityClass.logCoupon(c.Contact__c);

        // other logic below...
    }
}
```

The developer aims to track the **number of DML statements** to identify potential execution points at which the **DML limits** might be reached.

```
public class UtilityClass {

    public static void logCoupon(Id contactId) {

        Coupon_Log__c cl = new Coupon_Log__c();
        cl.Contact__c = contactId;

        insert cl;

        // other logic below...
    }
}
```

The helper class used by the trigger.

Tracking DML Example

There are **two techniques** that a developer can use to **track** DML statements: **filtering log messages** using the **Filter** option, and **hiding executed units** of other event types in the **Executed Units** tab.

The screenshot displays the Oracle SQL Developer interface with two main panels: the Execution Log and the Execution Overview.

Execution Log: This panel shows a list of database events. The 'Details' column contains information about each event. Three specific DML operations are highlighted with red boxes and arrows pointing to a yellow callout box:

- [6]Op:Insert(Type:Lead|Rows:1
- [13]Op:Insert(Type:Lead|Rows:1
- [20]Op:Insert(Type:Lead|Rows:1

A yellow callout box explains: "Enter 'DML', for example, to match log messages that contain 'DML'. Note that this filter is case-sensitive." Below the log, the 'Filter' checkbox is checked, and the text 'DML' is entered in the filter field.

Execution Overview: This panel shows a table of execution data. The 'Executed Units' tab is selected. The table has columns: What, Name, Sum, Avg, and Max. The data rows are:

What	Name	Sum	Avg	Max
Workflow	Assignment	0.13	0.04	0.06
Method	execute_ano...	214.22	214.22	214.22
Workflow	Workflow:Lead	19.64	6.55	10.94
Method	/services/dat...	214.27	214.27	214.27

A yellow callout box explains: "To display execution data for DML operations only, click all the buttons except the DML button." In the 'Show:' bar at the bottom, the 'DML' button is highlighted with a red box, while all other buttons (Methods, Queries, Workflow, Calls, Validations, Triggers, Pages) are unchecked.

Viewing an Execution Tree Example

A developer is working on a **custom feature** that involves invoking an **Apex method, triggers, workflow rules, and validation rules**.

```
public class AccountFeature {  
  
    public void generateAccounts(Integer maxRecords) {  
  
        List<Account> accounts = new List<Account>();  
        String accountName = '';  
  
        for (Integer i = 0; i < maxRecords; i++) {  
  
            Account a = new Account();  
            a.Name = generateName();  
            a.ISO_Code__c = generateISOCode();  
            a.Payment_Mode__c = generatePaymentMode();  
            accounts.add(a);  
        }  
  
        insert accounts;  
    }  
  
    // ...  
}
```

The developer intends to **analyze the execution path** to determine and understand the **processes** that are **involved**.

Viewing an Execution Tree Example

The **Stack Tree** panel of the Developer Console can be used to **trace** the **execution path** of a transaction. In this view, the **hierarchy** of **executed items** can be observed

The screenshot displays the Oracle Developer Console's **Stack Tree** and **Execution Log** panels. The **Stack Tree** panel on the left shows a hierarchical view of the execution path, with columns for **Unit**, **Duration**, and **Heap**. The **Execution Log** panel on the right shows a table of execution events with columns for **Timestamp**, **Event**, and **Details**.

Annotations with arrows point to specific items in the Stack Tree:

- Workflow rules** points to the **Payment Status Rule** and **Transaction Status Rule** under the **Workflow:Account** unit.
- Validation rule** points to the **Is_Valid_ISO_Code** unit under the **Validation:Account** unit.
- Apex trigger** points to the **Apex Trigger InefficientQueryLookupAccountTrigger** unit.

The **Execution Log** table contains the following data:

Timestamp	Event	Details
03:03:10:001	USER_INFO	[EXTERNAL] 0052v000007YeuUv jayandyar
03:03:10:001	EXECUTION_ST...	
03:03:10:001	CODE_UNIT_ST...	[EXTERNAL] execute_anonymous_apex
03:03:10:002	VARIABLE_SCO...	[1] Account true false
03:03:10:002	HEAP_ALLOCATE	[79] Bytes:3
03:03:10:002	HEAP_ALLOCATE	[84] Bytes:152
03:03:10:002	HEAP_ALLOCATE	[399] Bytes:408
03:03:10:002	HEAP_ALLOCATE	[412] Bytes:408
03:03:10:002	HEAP_ALLOCATE	[520] Bytes:48
03:03:10:002	HEAP_ALLOCATE	[139] Bytes:6
03:03:10:002	HEAP_ALLOCATE	[EXTERNAL] Bytes:5
03:03:10:002	STATEMENT_EX...	[1]
03:03:10:002	STATEMENT_EX...	[1]
03:03:10:002	HEAP_ALLOCATE	[1] Bytes:4
03:03:10:002	VARIABLE_ASSE...	[1] () 0x506de02a
03:03:10:002	STATEMENT_EX...	[1]

At the bottom of the console, there is a **Logs** panel with tabs for **Tests**, **Checkpoints**, **Query Editor**, **View State**, **Progress**, and **Problems**. The **Logs** panel shows a table of log entries:

User	Application	Operation	Time	Status	Read
Martin Gessner	Unknown	/services/data/v49.0/tooling...	25/09/2020, 03:03:10	Success	
Martin Gessner	Unknown	/services/data/v49.0/tooling...	25/09/2020, 02:53:39	Success	

Scenarios and Solutions

Table of Contents



Use Cases



SCENARIO

A developer who has just executed Apex code from the Execute Anonymous Window wants to know how many times a method in the code was called.



SOLUTION

This information can be obtained quickly by accessing the **Execution Overview** panel of the **Log Inspector**. The '**Executed Units**' tab of the panel shows the number of times each process was called during the transaction.



Use Cases



SCENARIO

An Apex trigger uses a helper class that contains an SOQL query. The developer who wrote the code for the helper class needs to know how many records are returned by the query.



SOLUTION

Information about the number of rows returned by an SOQL or SOSL query can be obtained from the '**Executed Units**' tab of the **Execution Overview** panel in the **Log Inspector**.

Use Cases



SCENARIO

A developer who is analyzing code performance needs to know the number of requests for SOQL rows as well as the total number of requests available for an Apex transaction.



SOLUTION

The '**Limits**' tab of the **Execution Overview** panel can be accessed to obtain information about the limit named 'SOQL_ROWS'. It would display the number of requests as well as the total number available.



Use Cases



SCENARIO

A developer seeking to improve the performance of a Visualforce page needs to know the amount of time taken by each process related to the execution of the page.



SOLUTION

The **Timeline** tab of the **Execution Overview** panel provides information about the time taken by each process in milliseconds. Different **Scale options** are available to change the time frame.

Use Cases



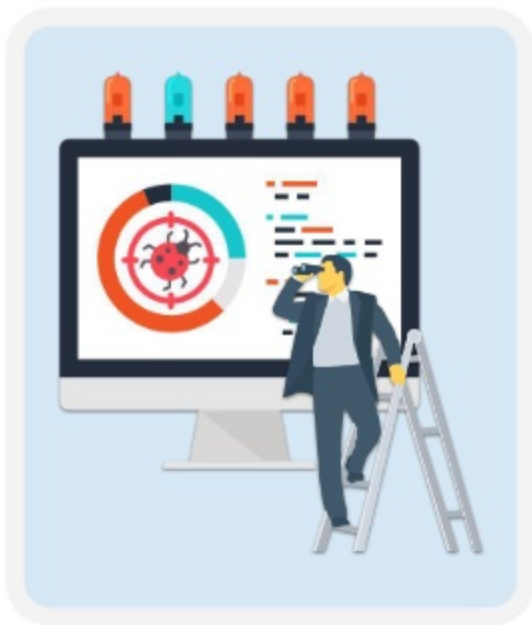
SCENARIO

A developer needs to debug JavaScript code associated with certain Lightning components. However, it is important to ensure that the debugging activity does not adversely influence the performance of Salesforce for other users in the org.



SOLUTION

It is possible to enable the debug mode for specific users in the org from the '**Debug Mode**' page in Setup. This ensures that performance is slower only for the users who are involved in the debugging activity.



Apex Tests

Table of Contents



Creating an Apex Test Class

Annotations

Test classes and test methods should be annotated with `@isTest`. The `(SeeAllData=true)` annotation can be added for org data access.

Start Test

Test code can be isolated from setup code with `Test.startTest()`. This method can be used to receive a fresh set of governor limits.

Stop Test

Asynchronous code can be forced to complete with `Test.stopTest()`. Original governor limits are restored after this point.

Create Test Data

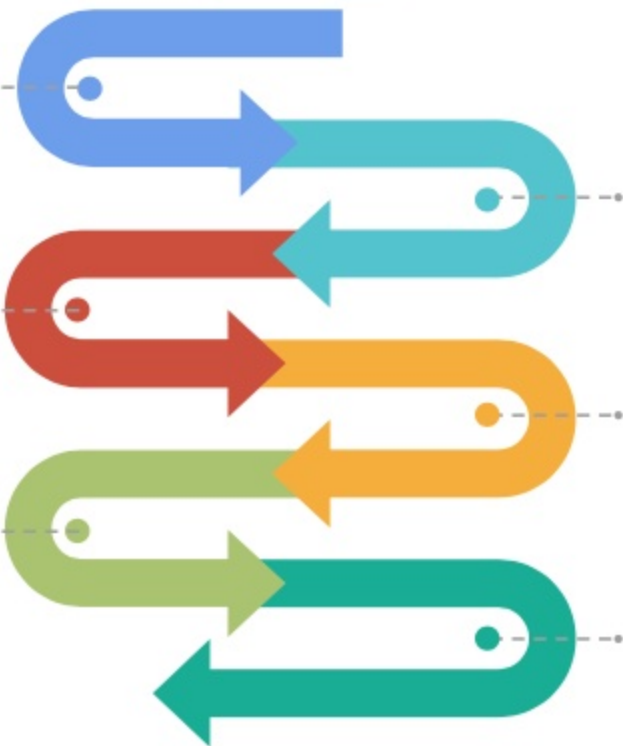
Test data should be created whenever possible. `Test.loadData` or a utility class can be used to create test data.

Code Execution

Code should execute all logic branches. Both single & bulk operations should be tested. Positive, negative and user-context test cases should be covered.

Assertions

Assert methods can be used to verify test results. Each test method should have at least one assertion. The more assertions, the better.



Apex Test Scenario / Solution I

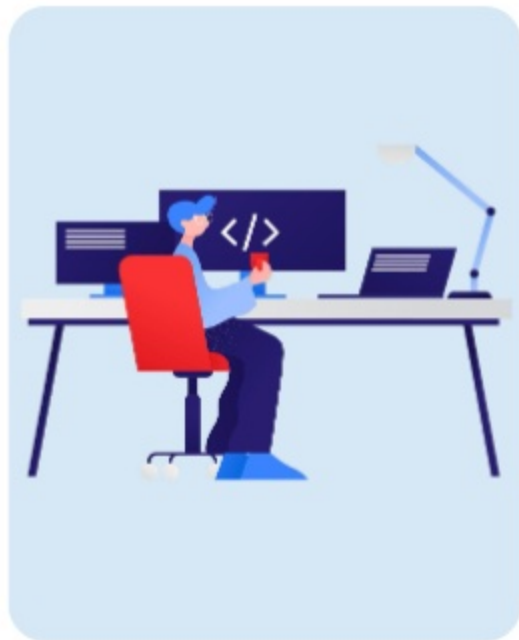
Table of Contents



Scenario I

◆ SCENARIO

An Apex class and its test class compile successfully in a development org. However, the deployment fails due to poor code coverage results in the production org. The developer responsible for the test class needs to increase the code coverage percentage. The Apex class and its corresponding test class are provided below.



Scenario I

This Apex class is used to determine the **discount** of a given amount based on the **number** of years a customer has been a **member**.

```
public class ItemDiscount {  
    public static Decimal calculateAmount(Integer yearsMember, Decimal price) {  
        Decimal amount = price;  
        if (price < 1000) { // discount rates if price is below 1000  
            if (yearsMember < 1) {  
                amount = price - (price * 0.10);  
            } else if (yearsMember >= 1 && yearsMember < 2) {  
                amount = price - (price * 0.12);  
            } else if (yearsMember >= 2 && yearsMember < 3) {  
                amount = price - (price * 0.14);  
            } else if (yearsMember >= 3 && yearsMember < 4) {  
                amount = price - (price * 0.18);  
            } else if (yearsMember >= 4 && yearsMember < 5) {  
                amount = price - (price * 0.2);  
            } else {  
                amount = price - (price * 0.25);  
            }  
        } else amount = price - 100; // a fixed discount value  
        return amount;  
    }  
}
```

ItemDiscount
Apex class

Scenario I

```
@isTest
private class ItemDiscountTest {
    @isTest
    static void testYears() {

        Decimal price = 750;
        Integer years = 2;
        Decimal amount = ItemDiscount.calculateAmount(years, price);
        System.assertEquals((price - (price * 0.14)), amount);
    }

    @isTest
    static void testAmount() {

        Decimal price = 1000;
        Integer years = 5;
        Decimal amount = ItemDiscount.calculateAmount(years, price);
        System.assertEquals((price - 100), amount);
    }
}
```

Test class for
ItemDiscount
Apex class

Solution I

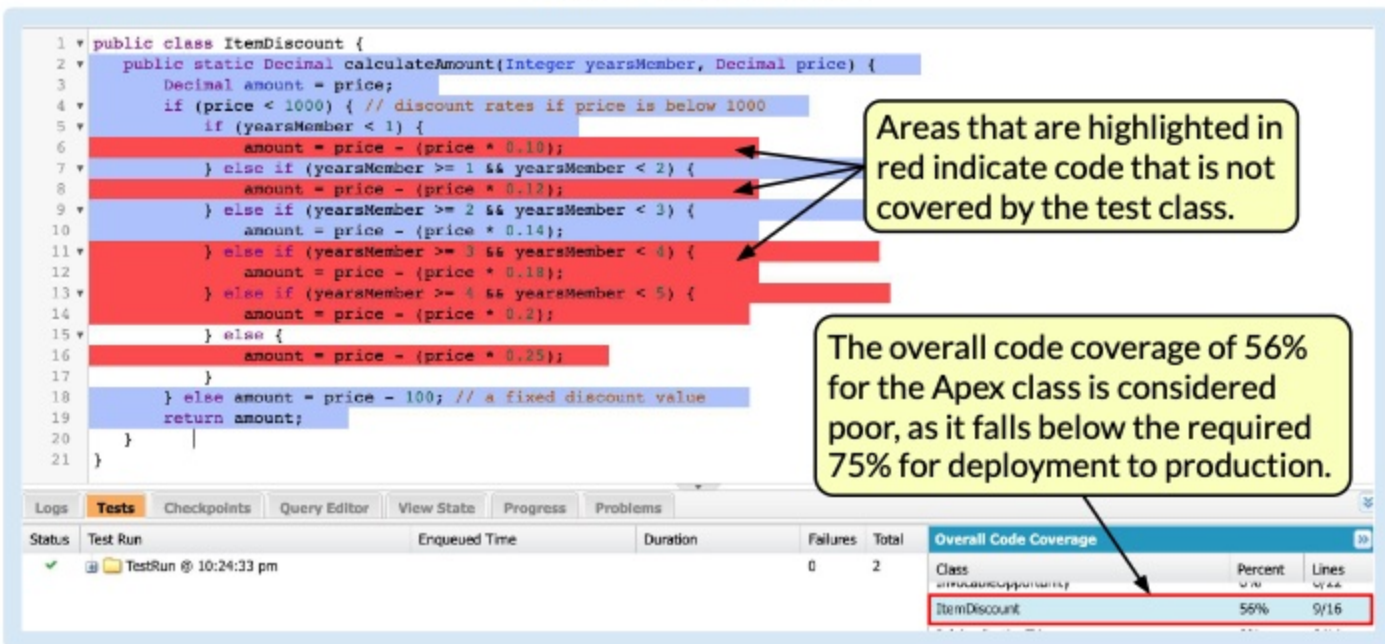


SOLUTION

In the *ItemDiscount* class, if the **price** is below 1,000, **discount rates** are applied depending on the **number of years** a customer has been a member. Otherwise, a **fixed discount value** is applied. In the test class, it only executes two scenarios - one in which the number of years is two, and the other in which the item price is 1,000. By updating the test class to cater to other input values, the code coverage can be improved accordingly.

Result of Unoptimized Test Class

The Apex test class can be run using the **Developer Console** to determine **code coverage results** as well as other details.



```
1 public class ItemDiscount {
2     public static Decimal calculateAmount(Integer yearsMember, Decimal price) {
3         Decimal amount = price;
4         if (price < 1000) { // discount rates if price is below 1000
5             if (yearsMember < 1) {
6                 amount = price - (price * 0.10);
7             } else if (yearsMember >= 1 && yearsMember < 2) {
8                 amount = price - (price * 0.12);
9             } else if (yearsMember >= 2 && yearsMember < 3) {
10                 amount = price - (price * 0.14);
11             } else if (yearsMember >= 3 && yearsMember < 4) {
12                 amount = price - (price * 0.18);
13             } else if (yearsMember >= 4 && yearsMember < 5) {
14                 amount = price - (price * 0.2);
15             } else {
16                 amount = price - (price * 0.25);
17             }
18         } else amount = price - 100; // a fixed discount value
19         return amount;
20     }
21 }
```

Areas that are highlighted in red indicate code that is not covered by the test class.

The overall code coverage of 56% for the Apex class is considered poor, as it falls below the required 75% for deployment to production.

Status	Test Run	Enqueued Time	Duration	Failures	Total	Overall Code Coverage		
✓	TestRun @ 10:24:33 pm			0	2	Class	Percent	Lines
						ItemDiscount	56%	9/16

Solution I



SOLUTION

Below is an example of how the `testYears()` **test method** can be improved to **execute all** the logic branches. A year-to-rate **map** is used to represent the year and its respective rate. A **for loop** is then used to iterate through the year values. A max rate is applied if the member reaches five years.

Solution I

Adding more **test data** to cover all possible **scenarios** greatly improves code **coverage** and **quality**.

```
@isTest
public class ItemDiscountTest {

    @isTest
    static void testYears() {

        Decimal price = 750;

        // rate used when member has reached 5 years and beyond
        Decimal maxRate = 0.25;

        // create a year-to-rate mapping
        Map<Integer, Decimal> rateMap = new Map<Integer, Decimal>{
            0 => 0.10, 1 => 0.12, 2 => 0.14, 3 => 0.18, 4 => 0.2
        };

        // loop through the years to execute all branches
        for (Integer years = 0; years <= rateMap.size(); years++) {

            Decimal amount = ItemDiscount.calculateAmount(years, price);
```

Continued...

Solution I

```
        Decimal rate = years >= rateMap.size() ? maxRate : rateMap.get(years);
        System.assertEquals((price - (price * rate)), amount);
    }
}

@Test
static void testAmount() {

    Decimal price = 1000;
    Integer years = 5;
    Decimal amount = ItemDiscount.calculateAmount(years, price);
    System.assertEquals((price - 100), amount);
}
}
```

Result of Optimized Test Class

Below is the **result** once the test class has been updated to **cover** all the **conditional branches**.

While only 75% of Apex code must be covered by tests to deploy the code in production, one should not focus on the percentage. It is a good practice to ensure that **every block** in the Apex class is covered, which ensures the **highest possible code coverage** that a unit test can achieve.

```
1 public class ItemDiscount {
2     public static Decimal calculateAmount(Integer yearsMember, Decimal price) {
3         Decimal amount = price;
4         if (price < 1000) { // discount rates if price is below 1000
5             if (yearsMember < 1) {
6                 amount = price - (price * 0.10);
7             } else if (yearsMember >= 1 && yearsMember < 2) {
8                 amount = price - (price * 0.12);
9             } else if (yearsMember >= 2 && yearsMember < 3) {
10                amount = price - (price * 0.14);
11            } else if (yearsMember >= 3 && yearsMember < 4) {
12                amount = price - (price * 0.18);
13            } else if (yearsMember >= 4 && yearsMember < 5) {
14                amount = price - (price * 0.2);
15            } else {
16                amount = price - (price * 0.25);
17            }
18        } else amount = price - 100; // a fixed discount value
19        return amount;
20    }
21 }
```

After optimizing the test class, the code coverage of the Apex class increases to 100%, which is more than sufficient for deployment to production.

Logs	Tests	Checkpoints	Query Editor	View State	Progress	Problems			
Status	Test Run	Enqueued Time		Duration	Failures	Total	Overall Code Coverage		
✓	TestRun @ 10:38:01 pm				0	2	Class	Percent	Lines
							InvocableOpportunity	0%	0/22
							ItemDiscount	100%	15/15
							InvocableAnniversaryTrainer	0%	0/11

Apex Test Scenario / Solution II

Table of Contents



Scenario II

◆ SCENARIO

An Apex trigger has been developed so that when a new Contact record is created and the Type field is set to "Member", it will generate a new Membership record to track membership status and details of the contact. Depending on the salary range of the contact, a certain Membership_Plan__c field value is also automatically assigned by the trigger. The feature was deployed to the production org. Later, it was noticed that the value of the Membership_Plan__c field on the generated membership records did not always match the salary range of the contacts.

After some troubleshooting, it was discovered that a field update action associated with a record-triggered flow was also being executed when a contact record was created. The issue was not encountered in the development org because it did not have the flow. The developer who created the original test class has been asked what could have been added to the test class to prevent this issue from occurring.



Scenario II

The following **test class** had been used to cover the **Apex trigger** code in a **previous deployment** to the production org.

```
// Original test class
@isTest
public class MembershipTest {

    @isTest
    static void testRegistration() {

        // Create a new contact
        Contact con = new Contact();
        con.FirstName = 'John';
        con.LastName = 'Smith';
        con.Salary_Range__c = 'A';
        con.Type__c = 'Member';
        insert con;

        // Check if a membership belongs to the contact
        List<Membership__c> results = [SELECT Id, Membership_Plan__c FROM Membership__c WHERE Contact__c = :con.Id];
        System.assertEquals(1, results.size()); // check that there is only one membership record for the contact
    }
}
```

Solution II



SOLUTION

It can be observed that the original test class does not utilize any **assertions** to verify the result. It only checks if a membership record was created, but **does not** check if the **membership plan** matches the **salary range** of the related contact. The following assertion could have been added after the first one:

```
// Check if membership plan matches with the salary range  
System.assertEquals(con.Salary_Range__c, results.get(0).Membership_Plan__c);
```

In general, test methods should contain **at least one assertion**. Good test methods include **more than one**. In addition, the test class would be **more efficient** if **more test methods** were added to test all the salary ranges and possible scenarios using both single and batch operations.

Apex Test Scenario / Solution III

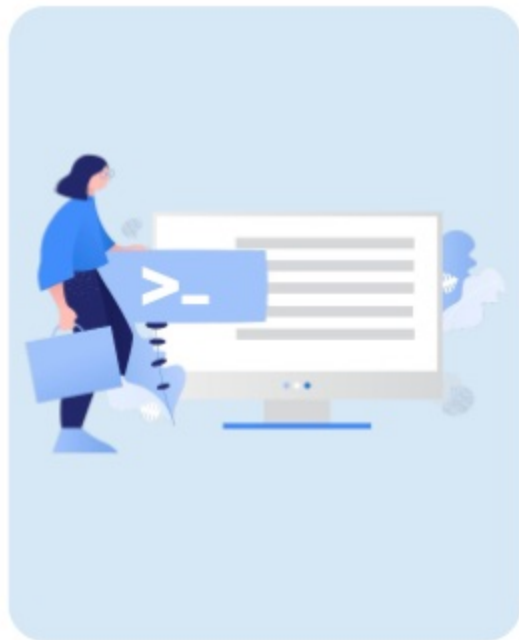
Table of Contents



Scenario III

◆ SCENARIO

A developer created an Apex trigger that updates a custom field (Vendor_Name__c) on the Account object when a related opportunity is won. A test class has been written to check and ensure that the behavior works as expected. In the sandbox org, the test code compiles successfully and the code coverage result is satisfactory. However, when it is deployed to the production org, the Apex test class fails. The developer is required to fix the test class to ensure that it does not fail.



Scenario III

The following **test class** is used to cover and deploy the **Apex trigger** to the production org.

```
// Test Class

@isTest(SeeAllData=true)
public class OpportunityTriggerTest {

    @isTest
    static void testMyTrigger() {

        Opportunity opp = [SELECT Id, Name, Project__r.Vendor_Name__c FROM Opportunity WHERE StageName = 'Negotiation/Review'
LIMIT 1];
        opp.StageName = 'Closed Won';
        update opp;

        Account acc = [SELECT Id, Vendor_Name__c FROM Account WHERE Id = :opp.AccountId LIMIT 1];
        System.assertEquals(acc.Vendor_Name__c, opp.Project__r.Vendor_Name__c);
    }
}
```

Solution III



SOLUTION

One likely cause is the **dependency** of the test class on **existing org data**, made evident by the `@isTest(seeAllData=true)` annotation. For example, the test scenario above may throw a **query exception** if no opportunities are found in the "Negotiation/Review" stage. Whenever possible, it is **recommended** to create **dedicated test data** for the test class.

Solution III



SOLUTION

Some important considerations related to the creation of test data are as follows:

◆ CREATE TEST DATA WITH TEST.LOADDATA

Static resources can be used to contain data for test classes. A **CSV file** containing test records can be uploaded as a static resource, which can be readily loaded into a test method:

```
List<sObject> records = Test.loadData(Opportunity.sObjectType, 'myStaticResourceName');
```

◆ CREATE TEST DATA WITH A TEST UTILITY CLASS

In general, **utility classes** are public classes that contain reusable code. Among other uses, a test utility class can be created to **generate data** for test classes. Below is an example class for creating test Account records.

Solution III

Shown below is an example **utility class**, sometimes called a **test data factory**, that is used for generating **test data** that can be shared by **multiple** test classes.

```
@isTest
public class MyTestDataFactory {
    public static void createAccounts(Integer numAccts) {

        List<Account> accts = new List<Account>();

        for (Integer i = 1; i <= numAccts; i++) {
            Account a = new Account(Name='Test Account ' + i);
            accts.add(a);
        }

        insert accts;
    }
}

// For example, to create 100 test accounts, run:
// MyTestFactory.createAccounts(100);
```

Solution III



SOLUTION

◆ CREATE TEST DATA IN A TEST SETUP METHOD

The **test setup method** contains the code that generates test data. Changes performed by one test method are **not carried** over to the next. Any changes performed by one method are **rolled back** after the method finishes executing. Note that **only one** test setup method can be used in a single test class.

```
@testSetup
static void setup() {

    // create 50 accounts
    MyTestFactory.createAccounts(50);
}
```

Apex Test Scenario / Solution IV

[Table of Contents](#) 

Scenario IV



◆ SCENARIO

A developer wrote a test class for a controller and noticed that at some point in the testing process, a limit exception is thrown. The test class needs to be modified to avoid hitting the governor limits.

Scenario IV

When the following **test class** is run, a **Limit Exception** is thrown.

```
@isTest
public class ControllerTest {

    @isTest
    public static void testMethodx() {

        // ...other logic here prior to the following...

        Account acc = new Account(Name='Testers Inc. ');
        insert acc; // create account

        List<Opportunity> opps = new List<Opportunity>();
        // create opportunities
        for (Integer i = 1; i <= 100; i++) {
            Opportunity opp = new Opportunity();
            opp.AccountId = acc.Id;
            opp.Name = 'Test Opportunity ' + i;
            opp.StageName = 'Analysis';
            opp.CloseDate = Date.newInstance(2019, 12, 31);
            insert opp; // create opportunity
        }
    }
}
```

Solution IV



SOLUTION

A **DML statement** should not be placed inside a **for loop**. The **insert** statement above should be utilized for a **bulk operation** instead of a single operation. For example, the **for loop** should add opportunity records to a **list variable**, and then a **single insert statement** can be applied to that list variable **after** the loop. This approach requires only **one** DML operation to insert the test opportunities. The **Test.startTest()** and **Test.stopTest()** methods can be used to ensure that any code between these two statements is executed under a fresh set of governor limits. Also, the example below illustrates the use of the limit methods, **Limits.getDmlStatements()** and **Limits.getLimitDmlStatements()**, to perform a check and **avoid** hitting the governor limit for DML statements.

Limits, startTest, and stopTest

The test class below demonstrates how **Limits methods** can be used to **determine** and **avoid** hitting the governor limits in an Apex transaction.

```
@isTest
public class ControllerTest {

    @isTest
    public static void testMethodx() {

        // other logic and processes before here...

        Test.startTest();

        // create account
        Account acc = new Account(Name='Testers Inc. ');
        insert acc;

        List<Opportunity> opps = new List<Opportunity>();

        // create opportunities
        for (Integer i = 1; i <= 200; i++) {

            Opportunity opp = new Opportunity();
```

A new set of governor limits are applied after this point.

Continued...

Limits, startTest, and stopTest

```
    opp.AccountId = acc.Id;
    opp.Name = 'Test Opportunity ' + i;
    opp.StageName = 'Analysis';
    opp.Amount = 0;
    opp.CloseDate = Date.newInstance(2019, 12, 31);

    if (Limits.getDmlStatements() < Limits.getLimitDmlStatements()) {

        insert opp; // create opportunity
        System.assert(opp.Amount == 0);
        System.debug(Limits.getDmlStatements() + ' <= ' + Limits.getLimitDmlStatements());

    } else System.debug('One more DML statement will throw an error!');
}

Test.stopTest();
// ...
}
```

Limit methods can be used to determine limit status.

Original limits are restored after this point.

Limit Exception Avoided

The code in this example stops performing a DML statement to **avoid exceeding** the governor limits of **150** statements.

Execution Log		
Timestamp	Event	Details
23:18:43:752	USER_DEBUG	[35] DEBUG 141 <= 150
23:18:43:898	USER_DEBUG	[35] DEBUG 142 <= 150
23:18:44:050	USER_DEBUG	[35] DEBUG 143 <= 150
23:18:44:216	USER_DEBUG	[35] DEBUG 144 <= 150
23:18:44:376	USER_DEBUG	[35] DEBUG 145 <= 150
23:18:44:526	USER_DEBUG	[35] DEBUG 146 <= 150
23:18:44:681	USER_DEBUG	[35] DEBUG 147 <= 150
23:18:44:841	USER_DEBUG	[35] DEBUG 148 <= 150
23:18:44:985	USER_DEBUG	[35] DEBUG 149 <= 150
23:18:45:140	USER_DEBUG	[35] DEBUG 150 <= 150
23:18:45:141	USER_DEBUG	[36] DEBUG One more DML statement will throw an error!

DML statement is skipped if DML limit has been reached. Thus, limit exception is avoided.

☐ This Frame ☐ Executable ☒ Debug Only ☐ Filter

Apex Test Scenario / Solution V

Table of Contents



Scenario V



SCENARIO

A test class has been developed for an Apex trigger that assigns a set of random coupons, which are created by the marketing head of the company, to a contact when the contact's Has_Coupons__c checkbox field is ticked during the creation process.



Scenario V

The following **test class** was used to **cover** and **deploy** the Apex trigger in a previous **deployment**.

```
@isTest(SeeAllData=true)
public class CouponTest {

    @isTest
    static void myTestMethod() {

        MyTestFactory.generateCoupons();

        Contact con = new Contact();
        con.FirstName = 'New';
        con.LastName = 'Member';
        con.Has_Coupons__c = true; // assign coupons after insert
        insert con;

        // now, check if coupons have been assigned to this contact
        List<Coupon__c> coupons = [SELECT Name FROM Coupon__c WHERE Contact__r.Id = :con.Id];
        System.assert(coupons.size() > 0);
    }
}
```

Scenario V



◆ SCENARIO

After the feature was rolled out in the production org, some users discovered that no coupons were linked to the records that they added in the org. After investigating, it was discovered that they did not have access to the coupon records created by the marketing head. Changes need to be made to the test class to catch and prevent the issue.

Solution V



SOLUTION

The `System.runAs()` method executes code in the **context** of **another user**. **Test methods** can access org metadata, such as **Users** and **Profiles**, even without using the `SeeAllData=true` annotation. This allows the test method to either **create a new user** or **select an existing one** in the org and run code as that user. In the example below, the test class leverages a utility class to **generate a test user** for execution.

Solution V

The following updated test class uses **System.runAs()** methods to simulate **user operations** accordingly in the org.

```
@isTest
public class CouponTest {

    @isTest
    static void myTestMethod1() {

        User adminUser = MyTestFactory.createAdminUser(); // create a user with admin profile

        System.runAs(adminUser){

            Contact con = new Contact();
            con.FirstName = 'New';
            con.LastName = 'Member';
            con.Has_Coupon__c = true;
            insert con;

            List<Coupon__c> coupons = [SELECT Name FROM Coupon__c WHERE Contact__r.Id = :con.Id];
            System.assert(coupons.size() > 0);

        }
    }
}
```

Continued...

Solution V

```
@isTest
static void myTestMethod2() {

    User marketingUser = MyTestFactory.createMarketingUser(); // create a user with marketing profile
    System.RunAs(marketingUser){

        Contact con = new Contact();
        con.FirstName = 'New';
        con.LastName = 'Member';
        con.Has_Coupon__c = true;
        insert con;

        List<Coupon__c> coupons = [SELECT Name FROM Coupon__c WHERE Contact__r.Id = :con.Id];
        System.assert(coupons.size() > 0);

    }
}
```

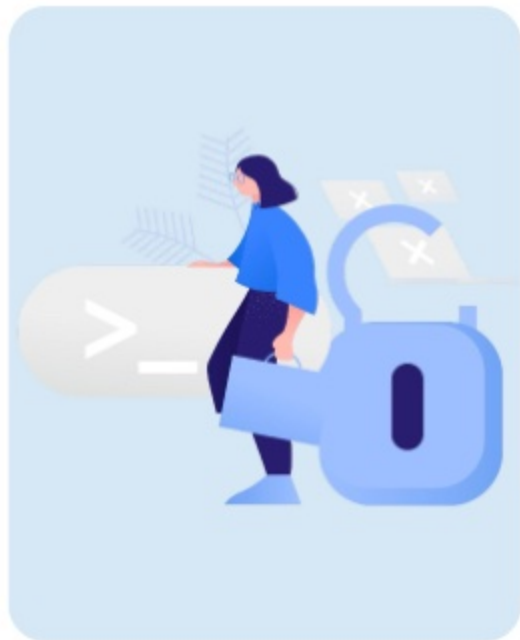
Apex Test Scenario / Solution VI

Table of Contents



Scenario VI

◆ **SCENARIO**
A developer has observed that when running certain test classes from the Salesforce user interface or Developer Console, the `UNABLE_TO_LOCK_ROW` error is sometimes encountered. The developer needs to determine the cause of the error and take the necessary step(s) to prevent its occurrence.



Solution VI



SOLUTION

The cause of the error is that tests executed from the **Salesforce user interface** or **Developer Console** run in **parallel** to speed up test run time. Data contention issues and *UNABLE_TO_LOCK_ROW* errors may occur due to the following:

◆ SAME RECORDS

Tests **update** the same records at the same time.

◆ DUPLICATE INDEX

Records are **created** with duplicate index field values.

Solution VI



SOLUTION

This can be prevented by deactivating **parallel test execution**. In *Setup > Custom Code > Application Test Execution > Options*, the 'Disable Parallel Apex Testing' checkbox can be selected to **deactivate** parallel test execution. The `@IsTest(isParallel=true)` annotation can be used on a test class to override disabled parallel testing.

Solution VI

Parallel Apex Testing can be **disabled** by going to *Setup > Custom Code > Application Test Execution > Options*.

The screenshot shows the Salesforce 'Application Test Execution' page. On the left sidebar, under 'Custom Code', 'Application Test Execution' is highlighted with a red box. A yellow callout box with a black border contains the text: 'Parallel Apex testing can be disabled to execute test methods one at a time and prevent the occurrence of **UNABLE_TO_LOCK_ROW** errors.' The main content area shows the 'Apex Test Execution Options' dialog box. In this dialog, the 'Disable Parallel Apex Testing' checkbox is checked and highlighted with a red box. Other options include 'Store Only Aggregated Code Coverage' and 'Independent Auto-Number Sequence'. The dialog has 'OK' and 'Cancel' buttons at the bottom.

Parallel Apex testing can be disabled to execute test methods one at a time and prevent the occurrence of **UNABLE_TO_LOCK_ROW** errors.

Apex Test Execution Options

- ☐ Store Only Aggregated Code Coverage
Stores code coverage results aggregated for all test methods; you can't view code coverage results for individual test methods. Use only to reduce code coverage calculation time when executing many tests on large volumes of code.
- ☒ **Disable Parallel Apex Testing**
Executes test methods one at a time.
- ☐ Independent Auto-Number Sequence
Ensures that auto-numbers in your organization's records don't have gaps due to test records created in Apex tests.

OK Cancel

Learn More

-  [A Guide to Application Performance Profiling in Force.com](#)
-  [Apex Developer Guide](#)
-  [Force.com Performance Profiling Using the Developer Console](#)
-  [Performance Tuning for Visualforce and Apex](#)
-  [Network Best Practices for Salesforce Architects](#)
-  [Debugging](#)
-  [Visualforce Performance: Best Practices](#)



Learn More

-  [Get Started With Analyzing Runtime Performance](#)
-  [Enable Debug Mode for Lightning Components](#)
-  [Testing Apex](#)
-  [Understanding Test Data](#)
-  [Testing Best Practices](#)
-  [Isolation of Test Data from Organization Data in Unit Tests](#)
-  [Order of Execution Overview \(Diagram\)](#)

